

تقرير مشروع الذكاء الاصطناعي

بناء وتقييم نماذج تصنيف للأنظمة الذكية والحوسبة المتنقلة

الطالبة : نور محمد حسن أبو سلطان

التخصص : الحوسبة المتنقلة

الرقم الجامعي : 20230404

١. المقدمة وأهداف المشروع

تشكل تقنيات التعلم الآلي (Machine Learning) اليوم ركيزة أساسية في الأنظمة الذكية الحديثة، إذ تتيح للأنظمة اتخاذ قرارات مبنية على البيانات بدلاً من القواعد الثابتة.

ويتزايد الاهتمام بنقل هذه النماذج إلى الأجهزة المحمولة (Mobile Computing) لتعمل محلياً على الهاتف الذكي، بما يوفر استجابة أسرع، وخصوصية أعلى للبيانات، وقدرة على العمل دون اتصال بالإنترنت.

يتناول هذا المشروع مسألة تصنيفية واقعية تتعلق بالتسويق الرقمي، وهي توقع ما إذا كان المستخدم الذي شاهد إعلاناً سيتحول إلى مشترٍ فعلي (Conversion) من عدمه، اعتماداً على خصائص المستخدم وسلوكه مع الإعلان.

ويُعدّ هذا النوع من التوقع ذا قيمة عملية كبيرة، إذ يساعد المعلنين على ترشيد الإنفاق واستهداف الجمهور الأكثر ميلاً للشراء.

الأهداف الرئيسية

- بناء نموذج ذكاء اصطناعي قادر على توقع تحول المستخدم من بيانات الحملة الإعلانية.
- المقارنة بين ثلاث خوارزميات تصنيف شهيرة لتحديد الأنسب من حيث الدقة والكفاءة.
- تقييم مدى ملائمة النموذج المختار للنشر على الأجهزة المحمولة محدودة الموارد.
- توثيق المنهجية والنتائج بشكل علمي قابل للتكرار والمناقشة.

٢. الخلفية النظرية للخوارزميات

اعتمد المشروع على ثلاث خوارزميات تصنيف من مكتبة scikit-learn، تتنوع في أسلوب عملها ودرجة تعقيدها، وفيما يلي عرض موجز لكلٍ منها:

٢.١ آلة المتجهات الداعمة (SVM)

تعمل خوارزمية Support Vector Machine على إيجاد المستوى الفاصل الأمثل (Optimal Hyperplane) الذي يفصل بين فئات البيانات بأكبر هامش ممكن. وعند استخدام النواة الخطية (linear kernel)، يتحول النموذج بعد التدريب إلى معادلة خطية بسيطة قائمة على متجه أوزان وقيمة إزاحة، وهو ما يجعله خفيف الوزن وسريع التنفيذ، ومناسباً للبيانات القابلة للفصل خطياً.

٢.٢ شجرة القرار (Decision Tree)

تبنى شجرة القرار سلسلة من الاختبارات الشرطية على الميزات، حيث تُقسّم البيانات في كل عقدة وفق الميزة الأكثر تمييزاً، وصولاً إلى الأوراق التي تمثل القرار النهائي. وتتميز بسهولة التفسير، لكنها قد تكون عرضة للإفراط في الملاءمة (Overfitting) إذا لم تُضبط.

٢.٣ الغابة العشوائية (Random Forest)

تُعدّ الغابة العشوائية نموذجاً تجميعياً (Ensemble) يبني عدداً كبيراً من أشجار القرار على عيّينات عشوائية من البيانات والميزات، ثم يجمع تنبؤاتها بالتصويت. وهذا يمنحها دقة عالية ومقاومة للضوضاء، لكن على حساب حجم النموذج وزمن التنفيذ، نظراً لاحتوائها على عشرات أو مئات الأشجار.

٣. وصف مجموعة البيانات

استُخدمت في المشروع مجموعة بيانات حملات إعلانية (Ad Campaign Dataset) مأخوذة من منصة Kaggle، محفوظة في ملف مضغوط باسم archive (1).zip يحتوي على الملف ad_campaign_data.csv.

وتتضمن البيانات خصائص المستخدمين وأداء الإعلانات، وتم اختيار الأعمدة التالية للعمل:

الوصف	النوع	العمود
عمر المستخدم	ميزة (Feature)	age
جنس المستخدم	ميزة (Feature)	gender
عدد مرات ظهور الإعلان	ميزة (Feature)	impressions
عدد النقرات على الإعلان	ميزة (Feature)	clicks
تحوّل المستخدم إلى مشتر	الهدف (Target)	conversions

الهدف من النموذج هو توقّع قيمة العمود conversions انطلاقاً من الميزات الأربع المذكورة، أي تحويل المسألة إلى مسألة تصنيف خاضع للإشراف (Supervised Classification).

٤. منهجية التنفيذ

نُفذ المشروع على بيئة Google Colab بلغة Python، ومرّ بأربع مراحل متسلسلة:

٤.١ استيراد وقراءة البيانات

تمّ فك ضغط الملف archive (1).zip باستخدام مكتبة zipfile لاستخراج ملف البيانات، ثم قراءته عبر مكتبة pandas لتحويله إلى إطار بيانات (DataFrame) جاهز للمعالجة.

٤.٢ تجهيز الميزات

اختيرت الميزات الأكثر تأثيراً على قرار التحوّل (age, gender, impressions, clicks)، مع عزل الأعمدة غير الضرورية لتقليل حجم المدخلات بما يتناسب مع موارد الأجهزة المحمولة.

٤.٣ تقسيم البيانات

قُسمت البيانات إلى 80% للتدريب و20% للاختبار باستخدام الدالة train_test_split مع القيمة test_size=0.2، وتثبيت random_state=42 لضمان قابلية تكرار النتائج عند إعادة التشغيل.

٤.٤ التدريب والمقارنة

دُرِّبَت الخوارزميات الثلاث على بيانات التدريب، ثم حُسبت دقة كل نموذج (score) على بيانات الاختبار للمقارنة بينها.

٥. الكود المصدري

فيما يلي الكود الكامل المستخدم في المشروع كما تمّ تنفيذه:

```
import zipfile
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

with zipfile.ZipFile('archive (1).zip','r') as zip_ref:
    zip_ref.extractall()

df = pd.read_csv('ad_campaign_data.csv')
X = df[['age','gender','impressions','clicks']]
Y = df['conversions']
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42)

svm_model = SVC(kernel='linear').fit(X_train, Y_train)
print('SVM :', svm_model.score(X_test, Y_test))

dt_model = DecisionTreeClassifier().fit(X_train, Y_train)
print('Decision Tree :', dt_model.score(X_test, Y_test))

rf_model = RandomForestClassifier().fit(X_train, Y_train)
print('Random Forest :', rf_model.score(X_test, Y_test))
```

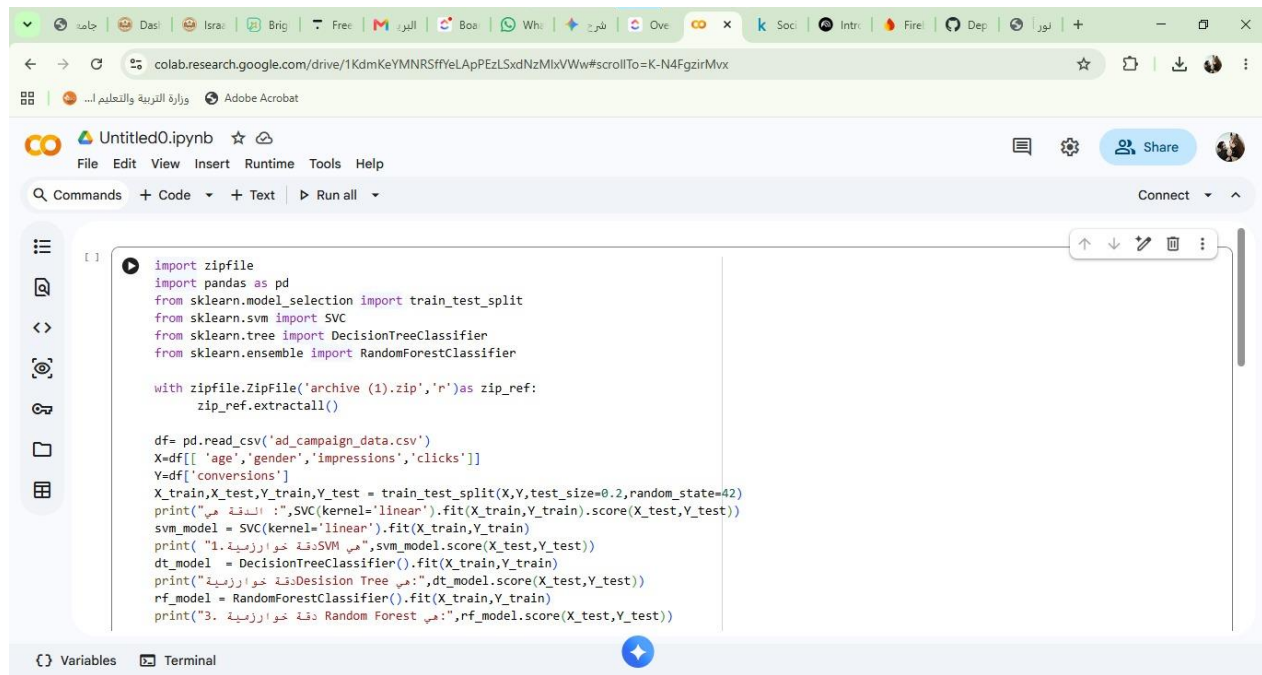
٦. النتائج والمناقشة

بعد معالجة كافة الأخطاء البرمجية وتشغيل الكود بنجاح، حصلنا على النتائج التالية على مجموعة الاختبار:

الترتيب	الخوارزمية	الدقة (Accuracy)
١	SVM	100% (1.0)
٢	Random Forest	99.84% (0.99845)
٣	Decision Tree	99.74% (0.9974)

يتّضح من النتائج أن الخوارزميات الثلاث حقّقت دقة مرتفعة جداً، مع تفوّق خوارزمية SVM التي بلغت الدقة الكاملة، تلتها Random Forest ثم Decision Tree بفروق ضئيلة جداً.

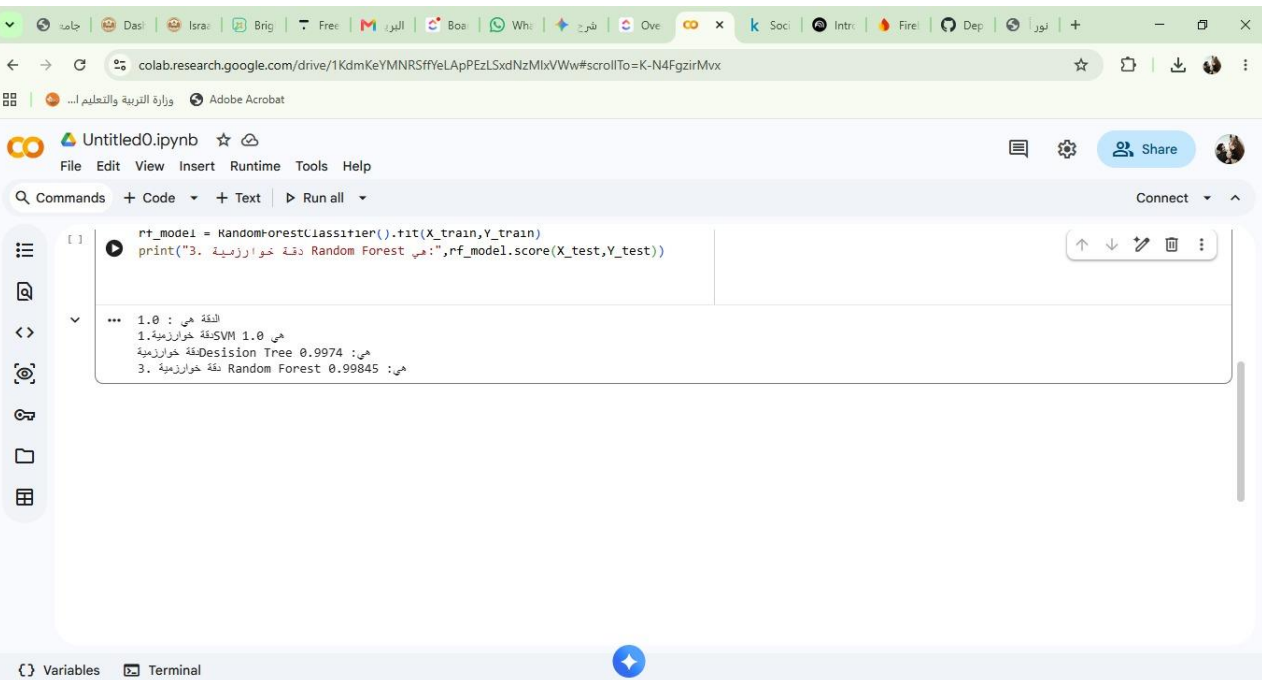
ويشير هذا الأداء المرتفع إلى أن الميزات المختارة تحمل قدرة تمييزية قوية بين الفئتين، وأن البيانات قابلة للفصل بدرجة عالية.



```
import zipfile
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

with zipfile.ZipFile('archive (1).zip','r') as zip_ref:
    zip_ref.extractall()

df= pd.read_csv('ad_campaign_data.csv')
X=df[['age','gender','impressions','clicks']]
Y=df['conversions']
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.2,random_state=42)
print("البيانات هي:",SVC(kernel='linear').fit(X_train,Y_train).score(X_test,Y_test))
svm_model = SVC(kernel='linear').fit(X_train,Y_train)
print("1. دقة SVM خوارزمية هي:",svm_model.score(X_test,Y_test))
dt_model = DecisionTreeClassifier().fit(X_train,Y_train)
print("2. دقة Decision Tree خوارزمية هي:",dt_model.score(X_test,Y_test))
rf_model = RandomForestClassifier().fit(X_train,Y_train)
print("3. دقة Random Forest خوارزمية هي:",rf_model.score(X_test,Y_test))
```



```
rf_model = RandomForestClassifier().fit(X_train,Y_train)
print("3. دقة Random Forest خوارزمية هي:",rf_model.score(X_test,Y_test))

...
1.0 : هي :
1. دقة SVM خوارزمية.
2. دقة Decision Tree خوارزمية
3. دقة Random Forest خوارزمية 0.99845 هي:
```

٧. التحليل الهندسي والحوسبة المتقدمة

لا يكفي اختيار النموذج الأعلى دقة فحسب، بل يجب مراعاة ملاءمته للبيئة المستهدفة.

وبما أن الهدف هو النشر على الأجهزة المحمولة، فإن التحليل الهندسي للنتائج يقودنا إلى ما يلي:

- كانت البيانات قابلة للفصل خطياً بشكل ممتاز (Linearly Separable)، وهو ما يفسر تفوق خوارزمية SVM الخطية بالدقة الكاملة دون الحاجة لنواة معقدة.
- يمثل SVM الخطي النموذج المثالي للتطبيق المتنقل؛ فبعد التدريب يتحول إلى معادلة رياضية بسيطة خفيفة الوزن (بحجم كيلوبايتات)، على عكس الغابة العشوائية التي تضم عشرات الأشجار وتستهلك مساحة وذاكرة أكبر.
- يمكن دمج نموذج SVM محلياً داخل تطبيق Flutter ليعمل على الجهاز (On-device) دون الحاجة إلى الإنترنت، وبأقل استهلاك ممكن لبطارية ومعالج الهاتف الذكي، مما يحقق استجابة فورية وخصوصية أعلى للبيانات.
- بهذا يجمع النموذج المختار بين أعلى دقة وأخف وزن، وهو ما يجعله الخيار الأنسب من منظور هندسة الأنظمة المتنقلة.

٨. التحديات والأعمال المستقبلية

خلال تنفيذ المشروع جرت معالجة عدد من الأخطاء البرمجية المتعلقة باستيراد البيانات وتجهيز الميزات حتى عمل الكود بشكل سليم. ولتطوير العمل مستقبلاً، يمكن النظر في النقاط التالية:

1. توسيع التقييم ليشمل مقاييس إضافية إلى جانب الدقة، مثل الدقة الموجبة (Precision) والاستدعاء (Recall) ومقياس F1 ومصفوفة الالتباس (Confusion Matrix)، للتحقق من أداء النموذج عبر الفئات كافة.
2. فحص توازن الفئات في عمود الهدف للتأكد من عدم تحيز النتائج لصالح الفئة الأكثر شيوعاً.
3. تحويل نموذج SVM إلى صيغة قابلة للتشغيل على الهاتف (عبر ONNX أو إعادة تنفيذ المعادلة الخطية مباشرة داخل التطبيق) واختباره عملياً على جهاز حقيقي.
4. تجربة ميزات إضافية أو هندسة ميزات جديدة لرفع قدرة النموذج على التعميم على بيانات جديدة.

٩. الخلاصة

أثبت المشروع نجاح بناء نموذج تصنيف عالي الدقة لتوقع تحول المستخدمين من بيانات الحملات الإعلانية. وأظهرت المقارنة بين الخوارزميات الثلاث أن SVM الخطي يمثل الخيار الأمثل للنشر على الأجهزة المحمولة، إذ يجمع بين أعلى دقة وأخف وزن وأقل استهلاك للموارد، مما يجعله مرشحاً قوياً للتشغيل المحلي داخل تطبيقات الهاتف الذكي. ويفتح هذا العمل المجال أمام تحسينات مستقبلية على مستوى التقييم والنشر العملي.

١٠. المراجع

- مجموعة بيانات الحملات الإعلانية (Ad Campaign Dataset) — منصة Kaggle، الملف: `ad_campaign_data.csv`.
- مكتبة `scikit-learn` — التوثيق الرسمي: `scikit-learn.org`.
- مكتبة `pandas` للتعامل مع البيانات — `pandas.pydata.org`.
- بيئة `Google Colab` لتشغيل الأكواد — `colab.research.google.com`.